

PROJECT 1: PERCEPTRON & ADALINE

EML 5808 Neural Networks, Fall 2026

ABSTRACT

We demonstrate the use of Perceptron and Adaline systems for simple character classification.

Christopher C Woodhouse
6246682

Introduction

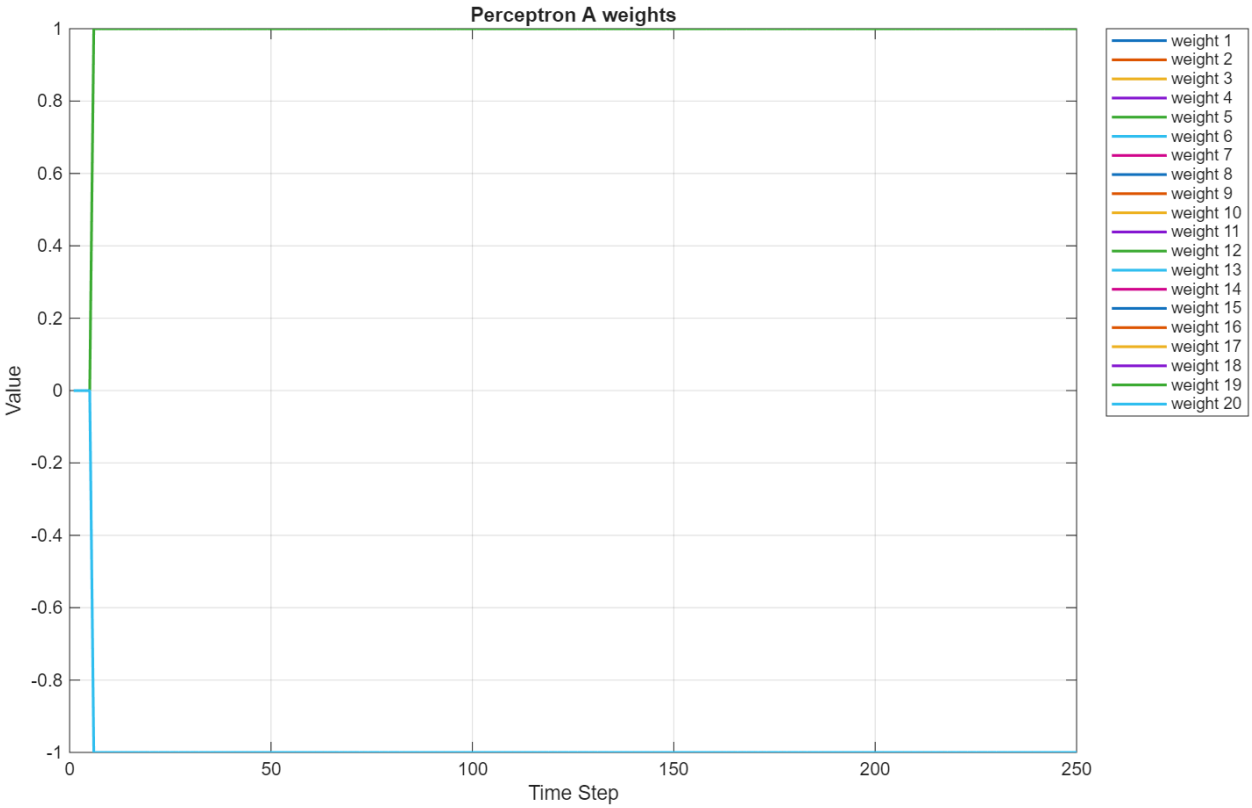
In this project we demonstrate the implementation of the Perceptron and Adaline algorithms.

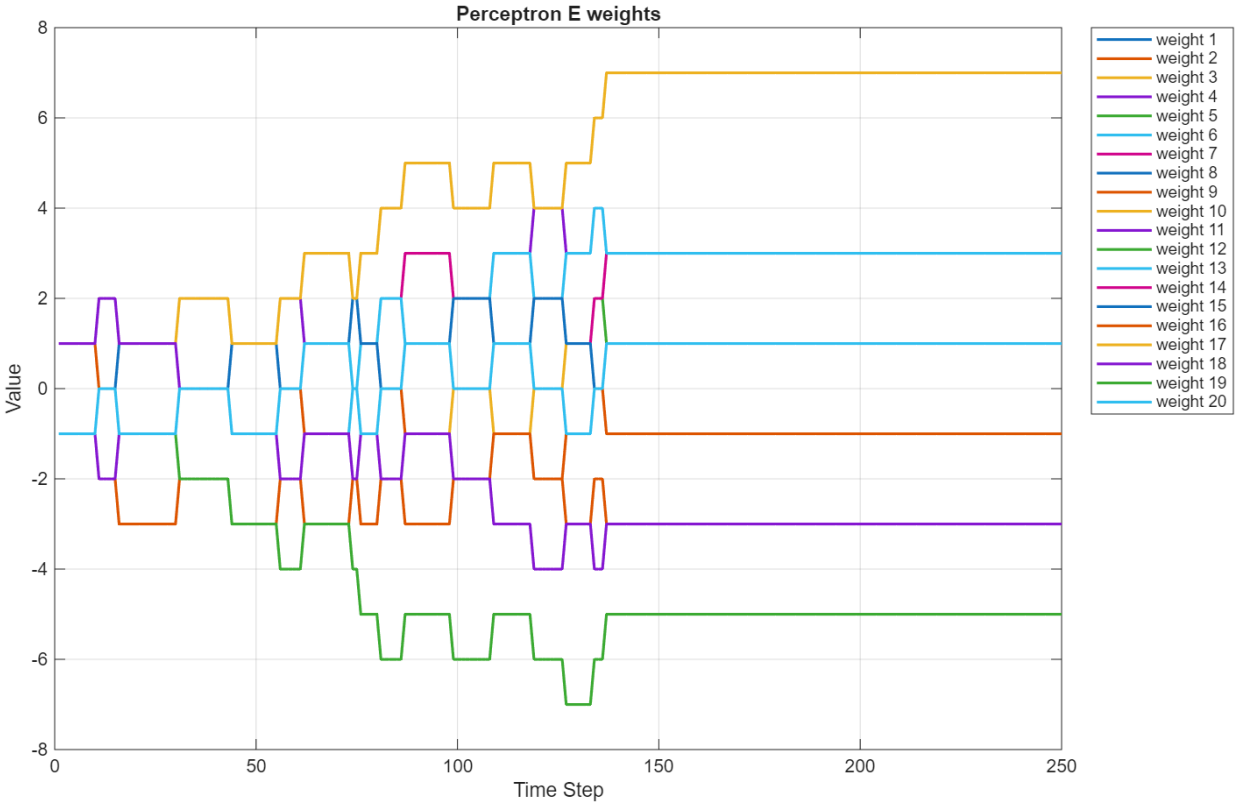
Our dataset is a set of 5-by-4 pixel patterns, each depicting one of five letters, A, E, I, O or U. We trained our models to either detect the A or E patterns. To do so we created 2 perceptrons and 6 adaline perceptrons with different learning rates. Of the Adaline perceptrons we went on to test with the best performing model, measured by mean squared error.

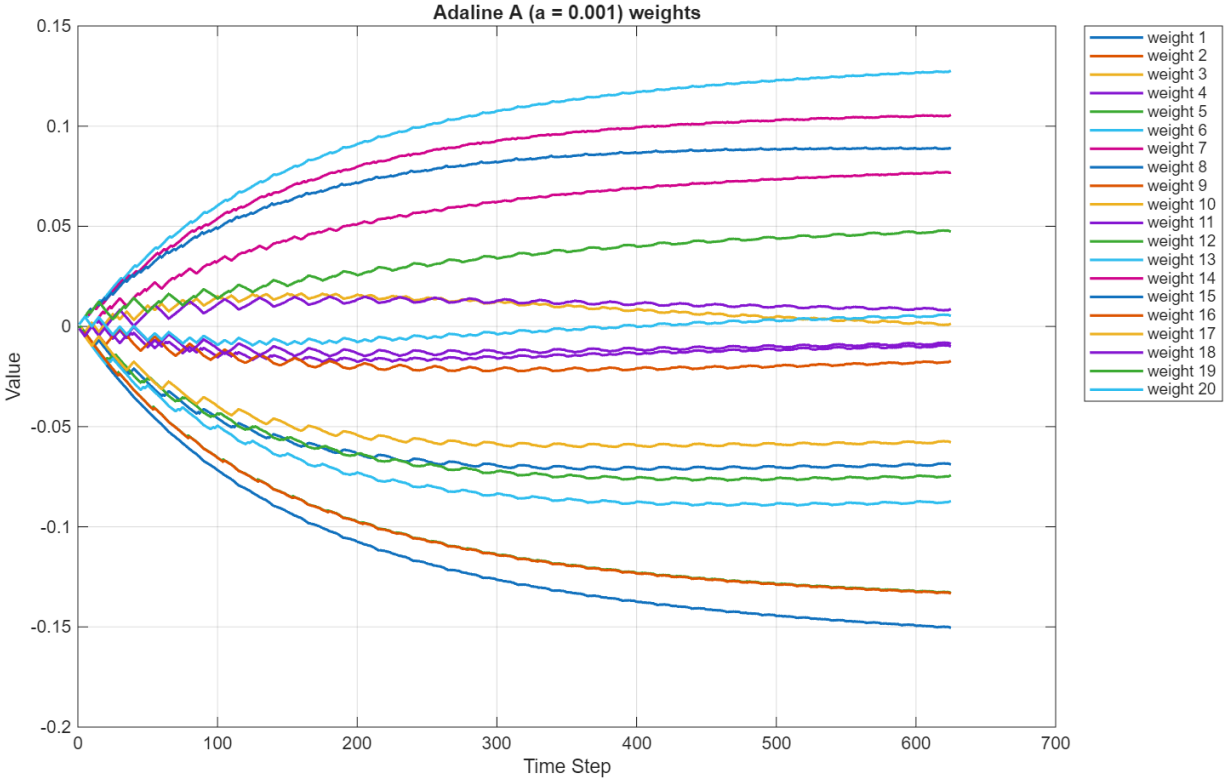
We tested our models on 3 increasingly difficult test data sets.

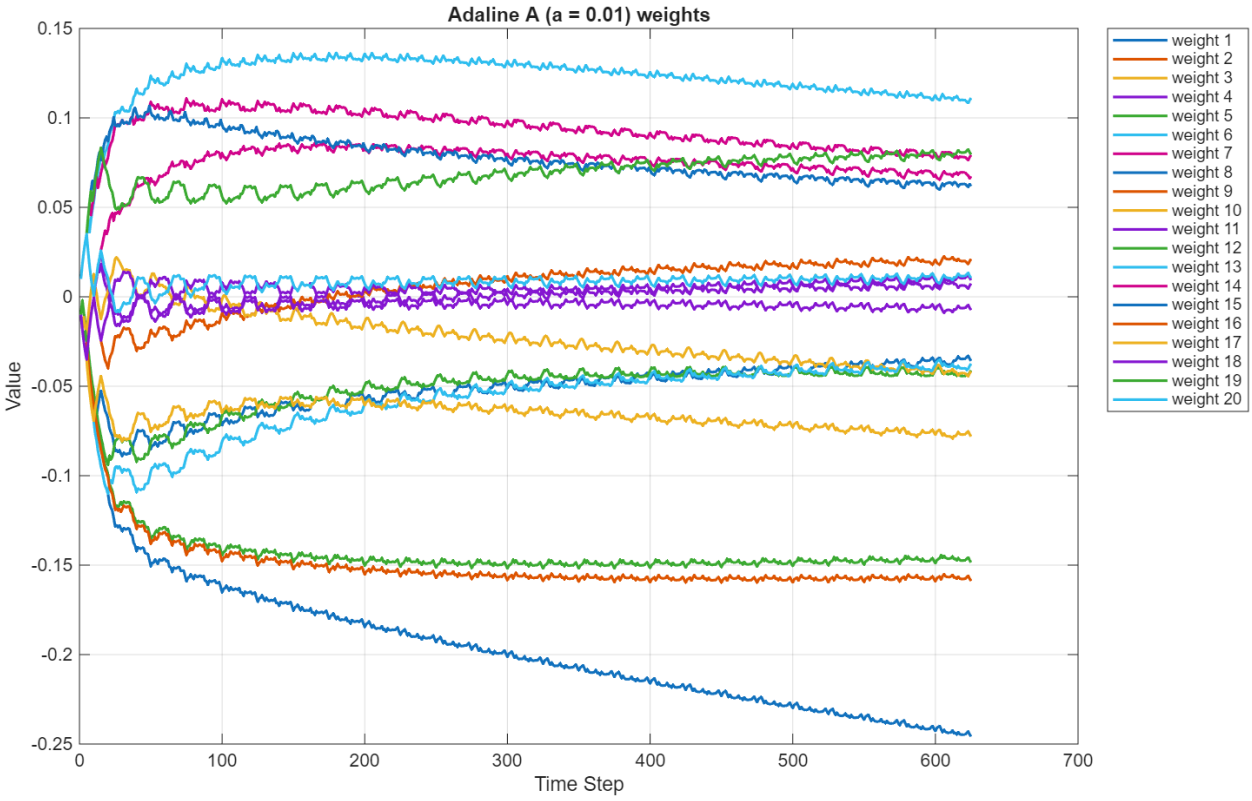
Data

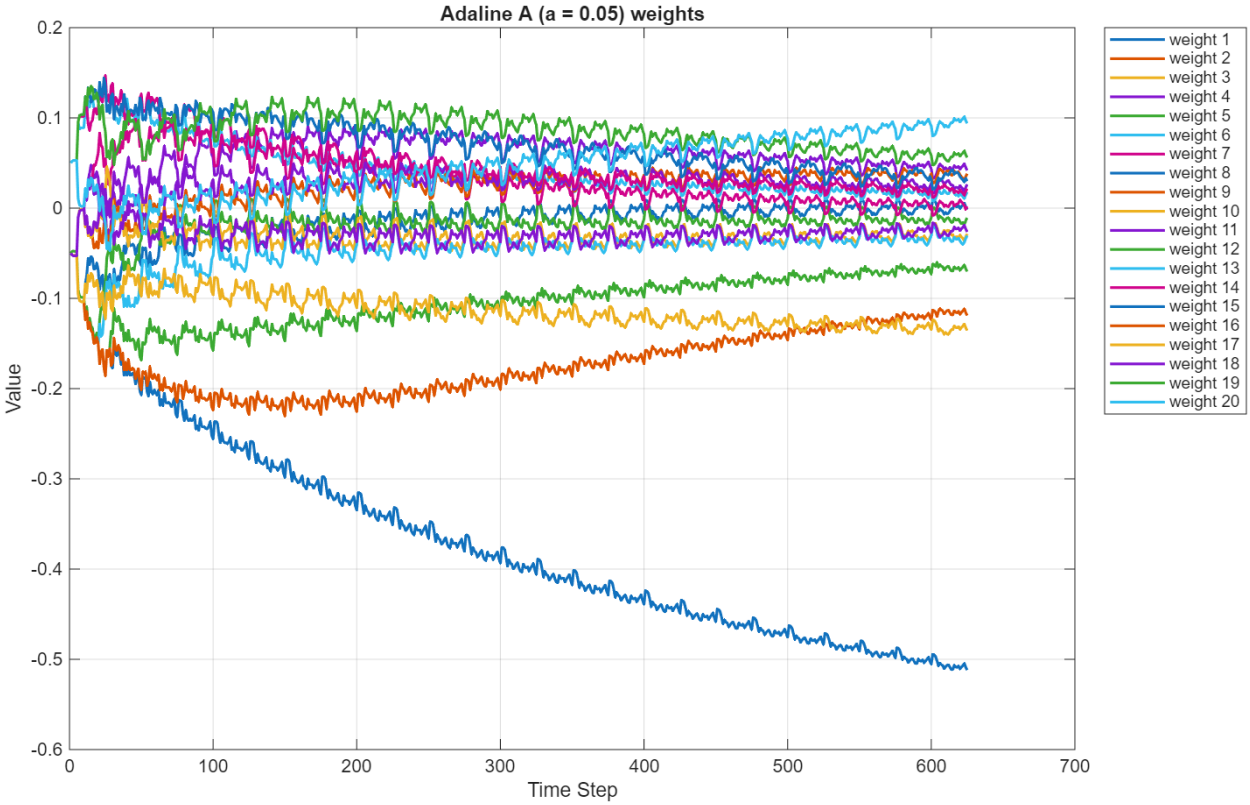
Weights Evolution

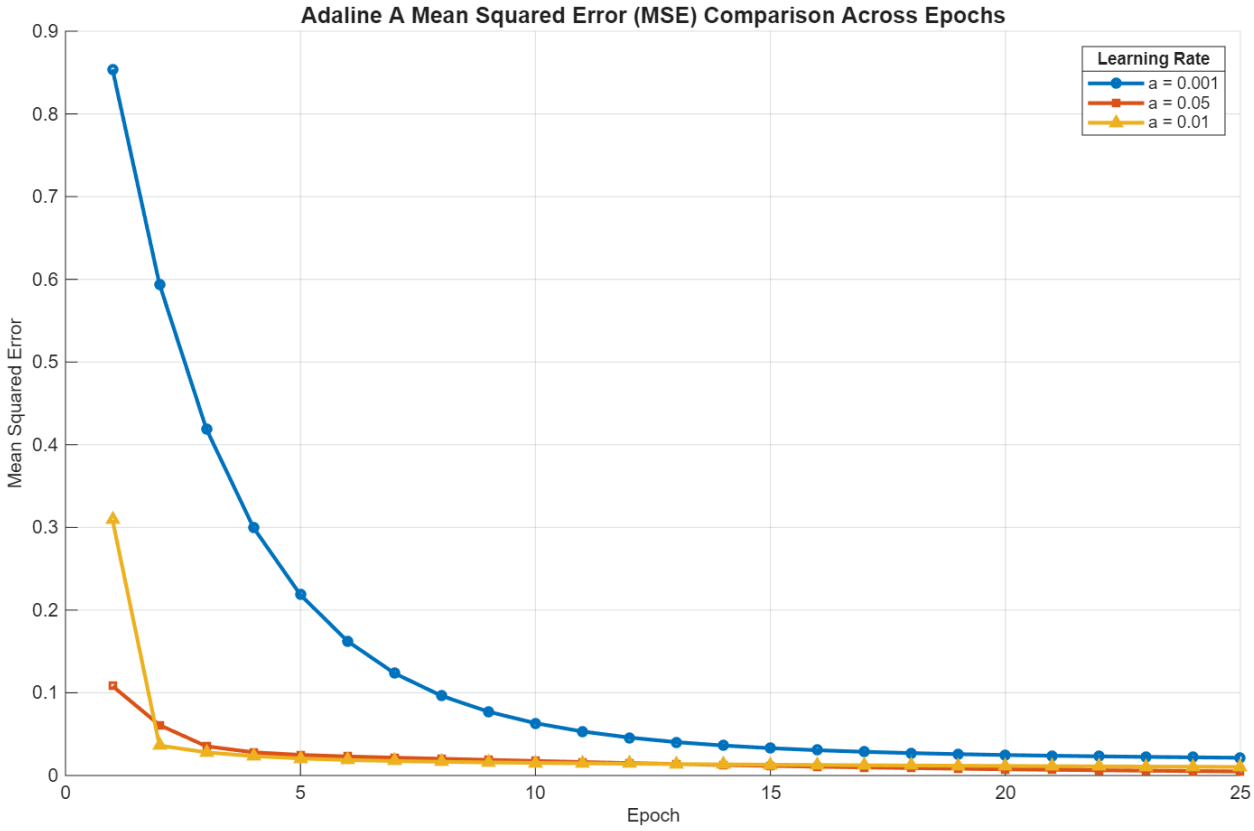


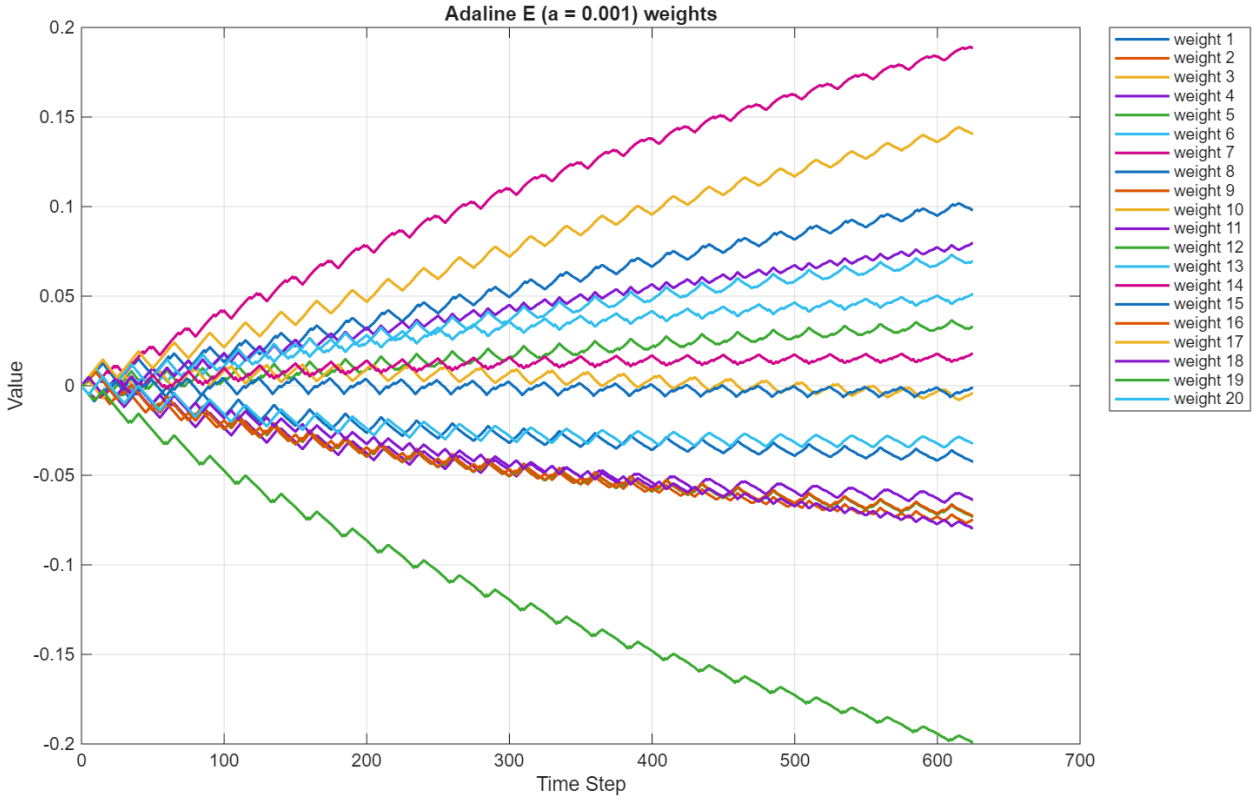

Perceptron E weights
Value
Time Step
8
6
4
2
0
-2
-4
-6
-8
0
50
100
150
200
250
weight 1
weight 2
weight 3
weight 4
weight 5
weight 6
weight 7
weight 8
weight 9
weight 10
weight 11
weight 12
weight 13
weight 14
weight 15
weight 16
weight 17
weight 18
weight 19
weight 20

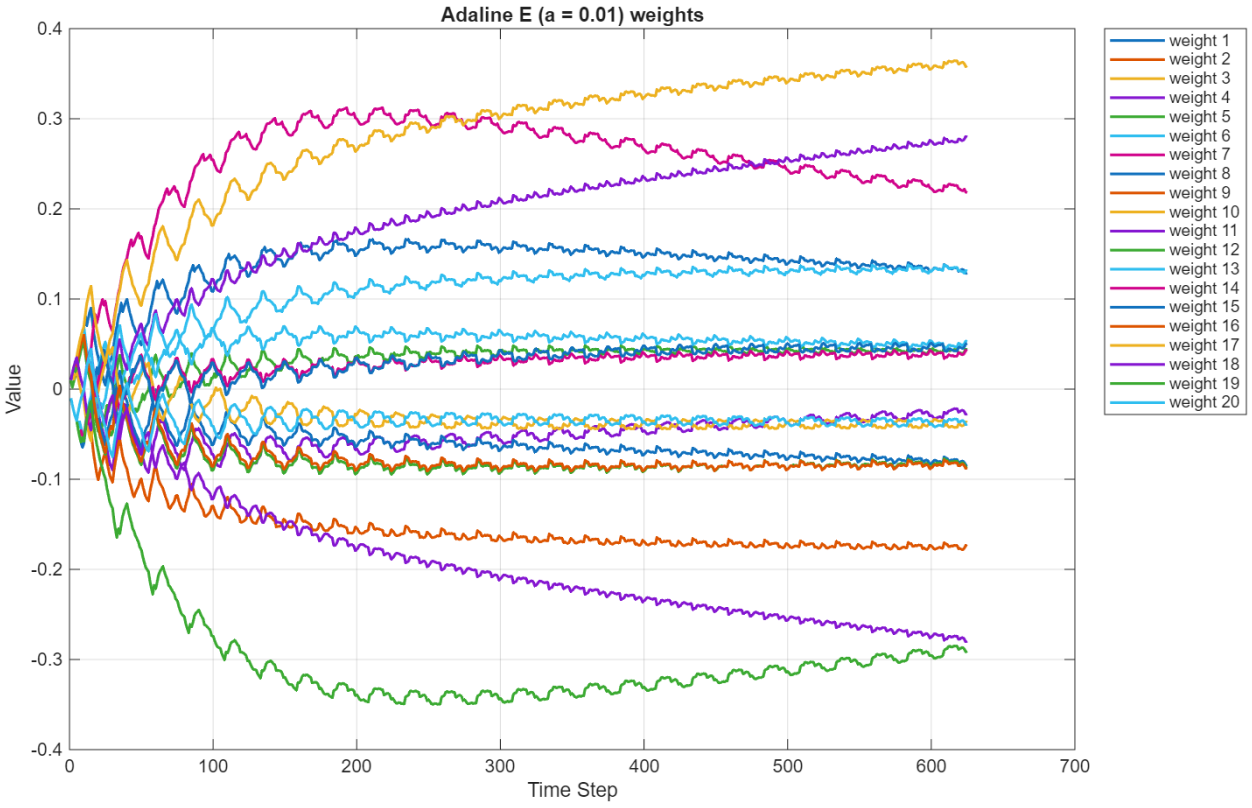

Adaline A (a = 0.001) weights
Value
Time Step
0.15
0.1
0.05
0
-0.05
-0.1
-0.15
-0.2
0
100
200
300
400
500
600
700
weight 1
weight 2
weight 3
weight 4
weight 5
weight 6
weight 7
weight 8
weight 9
weight 10
weight 11
weight 12
weight 13
weight 14
weight 15
weight 16
weight 17
weight 18
weight 19
weight 20

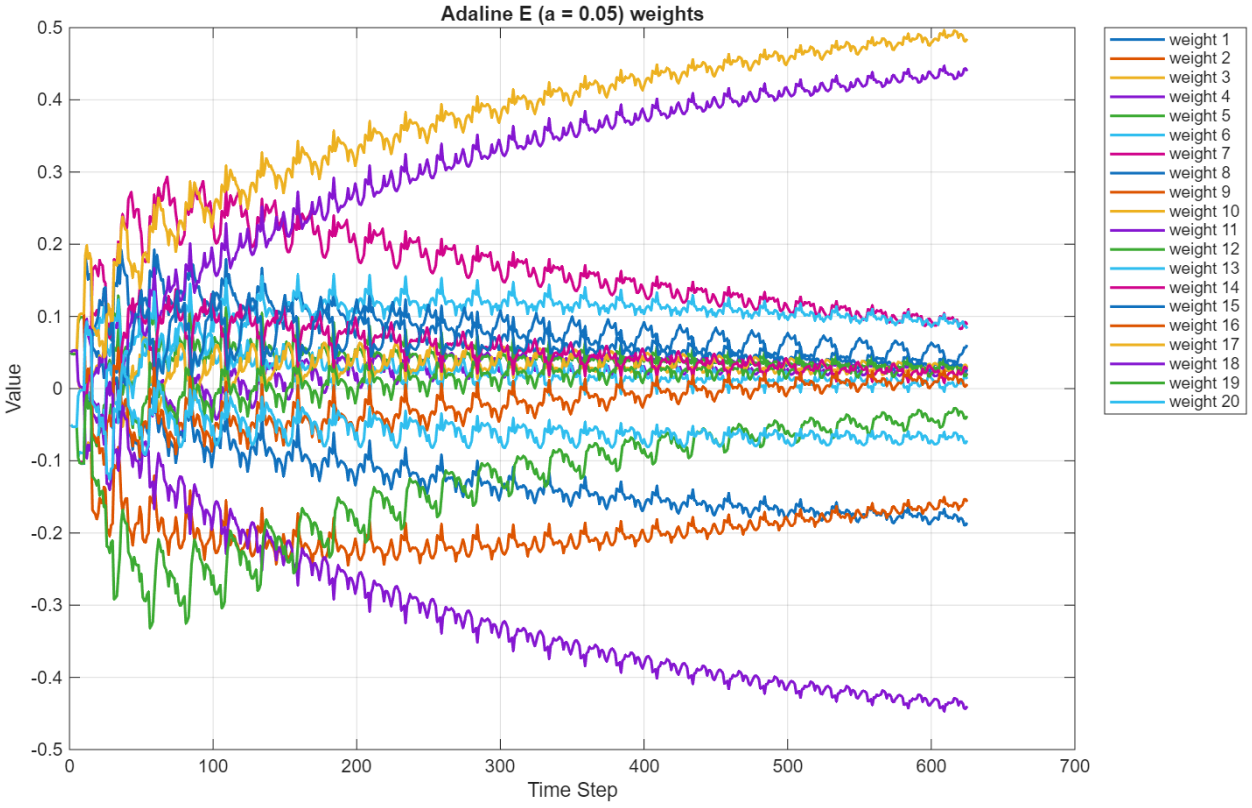

Adaline A (a = 0.01) weights
Value
Time Step
0.15
0.1
0.05
0
-0.05
-0.1
-0.15
-0.2
-0.25
0
100
200
300
400
500
600
700
weight 1
weight 2
weight 3
weight 4
weight 5
weight 6
weight 7
weight 8
weight 9
weight 10
weight 11
weight 12
weight 13
weight 14
weight 15
weight 16
weight 17
weight 18
weight 19
weight 20

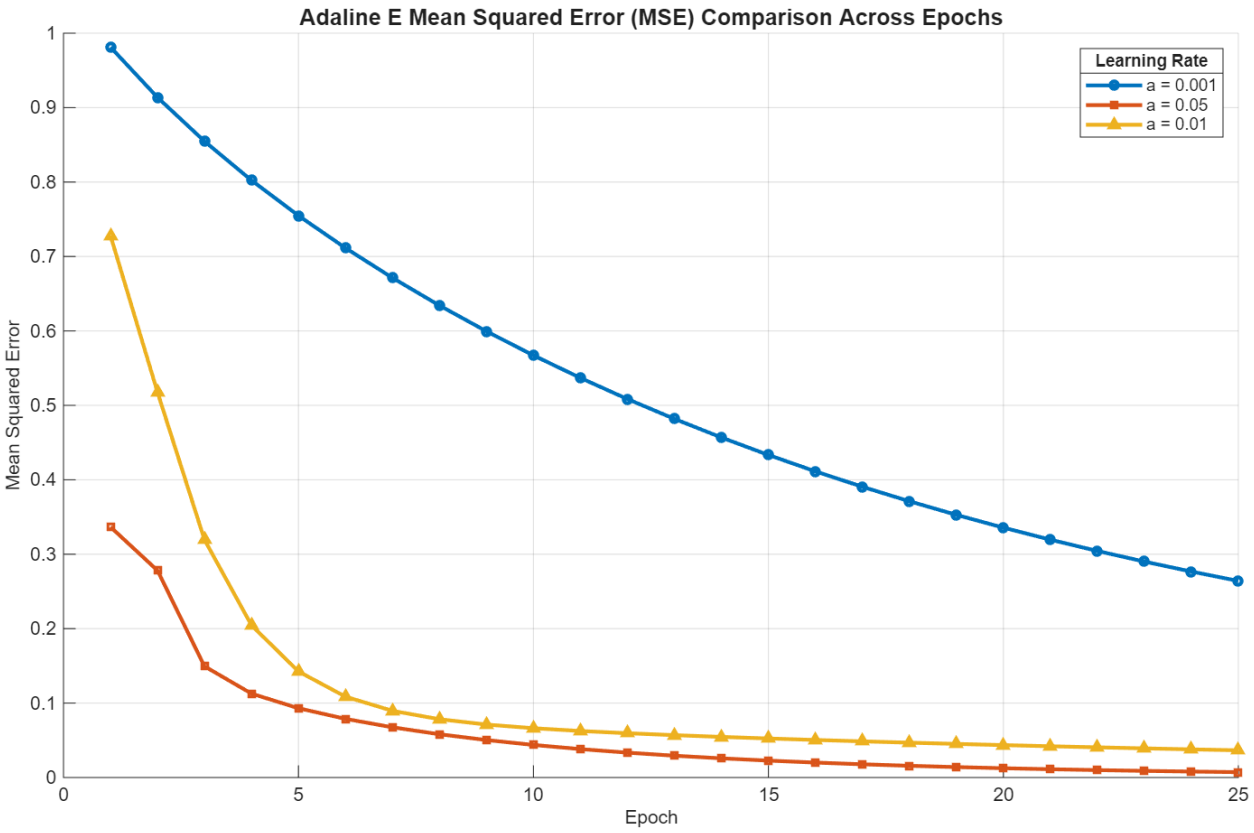

Adaline A (a = 0.05) weights
Value
Time Step
0.2
0.1
0
-0.1
-0.2
-0.3
-0.4
-0.5
-0.6
0
100
200
300
400
500
600
700
weight 1
weight 2
weight 3
weight 4
weight 5
weight 6
weight 7
weight 8
weight 9
weight 10
weight 11
weight 12
weight 13
weight 14
weight 15
weight 16
weight 17
weight 18
weight 19
weight 20


Adaline A Mean Squared Error (MSE) Comparison Across Epochs
Learning Rate
a = 0.001
a = 0.05
a = 0.01
Mean Squared Error
Epoch
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
0
5
10
15
20
25


Adaline E (a = 0.001) weights
weight 1
weight 2
weight 3
weight 4
weight 5
weight 6
weight 7
weight 8
weight 9
weight 10
weight 11
weight 12
weight 13
weight 14
weight 15
weight 16
weight 17
weight 18
weight 19
weight 20
Value
Time Step
0.2
0.15
0.1
0.05
0
-0.05
-0.1
-0.15
-0.2
0
100
200
300
400
500
600
700


Adaline E (a = 0.01) weights
Value
Time Step
0.4
0.3
0.2
0.1
0
-0.1
-0.2
-0.3
-0.4
0
100
200
300
400
500
600
700
weight 1
weight 2
weight 3
weight 4
weight 5
weight 6
weight 7
weight 8
weight 9
weight 10
weight 11
weight 12
weight 13
weight 14
weight 15
weight 16
weight 17
weight 18
weight 19
weight 20


Adaline E (a = 0.05) weights
Value
Time Step
0.5
0.4
0.3
0.2
0.1
0
-0.1
-0.2
-0.3
-0.4
-0.5
0
100
200
300
400
500
600
700
weight 1
weight 2
weight 3
weight 4
weight 5
weight 6
weight 7
weight 8
weight 9
weight 10
weight 11
weight 12
weight 13
weight 14
weight 15
weight 16
weight 17
weight 18
weight 19
weight 20



Test Data

A Perceptron

Row	p01	p02	p03	p04	p05	p06	p07	p08	p09	p10	p11	p12	p13	p14	p15	p16	p17	p18	p19	p20	p21	p22	p23	p24	p25	Hit Rate
Test 1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Test 2	1	1	1	1	-1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0.96
Test 3	1	1	1	1	-1	1	1	1	1	1	1	1	1	-1	1	1	1	1	1	1	1	1	1	1	1	0.92

E Perceptron

Row	p01	p02	p03	p04	p05	p06	p07	p08	p09	p10	p11	p12	p13	p14	p15	p16	p17	p18	p19	p20	p21	p22	p23	p24	p25	Hit Rate
Test 1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1	1	1	1	0.96
Test 2	1	1	1	1	-1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1	1	-1	1	1	1	0.88
Test 3	1	1	1	1	-1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	-1	-1	1	1	0.88

A Adaline

Row	p01	p02	p03	p04	p05	p06	p07	p08	p09	p10	p11	p12	p13	p14	p15	p16	p17	p18	p19	p20	p21	p22	p23	p24	p25	Hit Rate
Test 1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Test 2	1	1	1	1	1	1	1	1	1	1	1	1	-1	1	1	1	1	1	1	-1	1	1	1	1	1	0.92
Test 3	1	1	1	-1	1	1	-1	1	1	1	1	1	1	-1	1	1	1	1	1	1	1	1	1	1	1	0.88

E Adaline

Row	p01	p02	p03	p04	p05	p06	p07	p08	p09	p10	p11	p12	p13	p14	p15	p16	p17	p18	p19	p20	p21	p22	p23	p24	p25	Hit Rate
Test 1	1	1	1	1	1	1	1	-1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0.96
Test 2	1	1	1	1	-1	1	1	-1	1	-1	1	1	1	1	1	1	1	1	1	-1	1	1	1	1	1	0.84
Test 3	1	1	1	1	-1	1	1	-1	1	-1	1	1	-1	1	1	1	1	1	1	-1	1	1	1	1	1	0.8

Conclusion

My testing led to several interesting and unexpected results. I was surprised how accurately a single perceptron could come to learn the shape of letters, even if they are very low resolution.

I was also surprised to learn that the “A Detector” perceptron was able to learn the data in a single training step. I suspect this is due to the unique shape of the letter A. This model also had the best test performance of this project. This demonstrates that more complex architecture is not necessarily better than simply ones.

Appendix

Main project function

```
function [] = project1()



  % create "A Detector" perceptron
  ptron_A_weights = zeros(1,20, "double");
  ptron_A_bias = 0;
  % create "E Detector" perceptron
  ptron_E_weights = zeros(1,20, "double");
  ptron_E_bias = 0;

  % create "A Detector" adaline perceptron
  aline_A_weights = zeros(1,20, "double");
  aline_A_bias = 0;
  % create "E Detector" adaline perceptron
  aline_E_weights = zeros(1,20, "double");
  aline_E_bias = 0;
```

```
  % learning rates Adaline
  learningrateA1 = 0.001;
  learningrateA2 = 0.05;
  learningrateA3 = 0.01;

  learningrateE1 = 0.001;
  learningrateE2 = 0.05;
  learningrateE3 = 0.01;

  % load input & test data
  load("PP-1.mat");
  load("TA-1.mat");
  load("TE-1.mat");

  load("TST1-1.mat");
  load("TST2-1.mat");
  load("TST3-1.mat");




  % train neurons
  [ptron_A_weights, ptron_A_bias] = train_perceptron(ptron_A_weights, ptron_A_bias,
PP, TA, 10, 0.5, 'Perceptron A weights')
  [ptron_E_weights, ptron_E_bias] = train_perceptron(ptron_E_weights, ptron_E_bias, PP,
TE, 10, 0.5, 'Perceptron E weights')

  % adaline A
  [aline_A1_weights, aline_A1_bias, aline_A1_mse] = train_adaline(aline_A_weights,
aline_A_bias, PP, TA, 25, learningrateA1, 'Adaline A')
  [aline_A2_weights, aline_A2_bias, aline_A2_mse] = train_adaline(aline_A_weights,
aline_A_bias, PP, TA, 25, learningrateA2, 'Adaline A')
  [aline_A3_weights, aline_A3_bias, aline_A3_mse] = train_adaline(aline_A_weights,
aline_A_bias, PP, TA, 25, learningrateA3, 'Adaline A')

  graph_meansquarederrors(aline_A1_mse, learningrateA1, aline_A2_mse,
learningrateA2, aline_A3_mse, learningrateA3, "A")

  % adaline e
  [aline_E1_weights, aline_E1_bias, aline_E1_mse] = train_adaline(aline_E_weights,
aline_E_bias, PP, TE, 25, learningrateE1, 'Adaline E')
  [aline_E2_weights, aline_E2_bias, aline_E2_mse] = train_adaline(aline_E_weights,
aline_E_bias, PP, TE, 25, learningrateE2, 'Adaline E')
```

```
  [aline_E3_weights, aline_E3_bias, aline_E3_mse] = train_adaline(aline_E_weights,
aline_E_bias, PP, TE, 25, learningrateE3, 'Adaline E')

  graph_meansquarederrors(aline_E1_mse, learningrateE1, aline_E2_mse,
learningrateE2, aline_E3_mse, learningrateE3, "E")



  % decide which vesion of A and E adaline perceptrons are the best
  % referencing the graphs, 0.05 seems to have the best performance



  % test perceptrons
  % first 5 are A, second 5 are E
  A_truth = [1, 1, 1, 1, 1,  -1, -1, -1, -1, -1,  -1, -1, -1, -1, -1,  -1, -1, -1, -1, -1,  -1, -1, -1, -1, -1,];
  E_truth = [-1, -1, -1, -1, -1,  1, 1, 1, 1, 1,  -1, -1, -1, -1, -1,  -1, -1, -1, -1, -1,  -1, -1, -1, -1, -1,];

  % a perceptron
  [ATST1_results, ATST1_hitrate] = perceptron_test(ptron_A_weights, ptron_A_bias, TST1,
A_truth, false);
  [ATST2_results, ATST2_hitrate] = perceptron_test(ptron_A_weights, ptron_A_bias, TST2,
A_truth, false);
  [ATST3_results, ATST3_hitrate] = perceptron_test(ptron_A_weights, ptron_A_bias, TST3,
A_truth, false);

  test_table_display(ATST1_results, ATST1_hitrate, ATST2_results, ATST2_hitrate,
ATST3_results, ATST3_hitrate, 'ptron_A_testResults.xlsx');



  % e perceptron
  [ETST1_results, ETST1_hitrate] = perceptron_test(ptron_E_weights, ptron_E_bias, TST1,
E_truth, false);
  [ETST2_results, ETST2_hitrate] = perceptron_test(ptron_E_weights, ptron_E_bias, TST2,
E_truth, false);
  [ETST3_results, ETST3_hitrate] = perceptron_test(ptron_E_weights, ptron_E_bias, TST3,
E_truth, false);

  test_table_display(ETST1_results, ETST1_hitrate, ETST2_results, ETST2_hitrate,
ETST3_results, ETST3_hitrate, 'ptron_E_testResults.xlsx');



  % a adaline
  [alineATST1_results, alineATST1_hitrate] = perceptron_test(aline_A2_weights,
aline_A2_bias, TST1, A_truth, true);
```

```
  [alineATST2_results, alineATST2_hitrate] = perceptron_test(aline_A2_weights,
aline_A2_bias, TST2, A_truth, true);
  [alineATST3_results, alineATST3_hitrate] = perceptron_test(aline_A2_weights,
aline_A2_bias, TST3, A_truth, true);

  test_table_display(alineATST1_results, alineATST1_hitrate, alineATST2_results,
alineATST2_hitrate, alineATST3_results, alineATST3_hitrate, 'aline_A_testResults.xlsx');


  % e adaline
  [alineETST1_results, alineETST1_hitrate] = perceptron_test(aline_E2_weights,
aline_E2_bias, TST1, E_truth, true);
  [alineETST2_results, alineETST2_hitrate] = perceptron_test(aline_E2_weights,
aline_E2_bias, TST2, E_truth, true);
  [alineETST3_results, alineETST3_hitrate] = perceptron_test(aline_E2_weights,
aline_E2_bias, TST3, E_truth, true);

  test_table_display(alineETST1_results, alineETST1_hitrate, alineETST2_results,
alineETST2_hitrate, alineETST3_results, alineETST3_hitrate, 'aline_E_testResults.xlsx');
```

```
% perceptron training
function [weights, bias] = train_perceptron(init_weights, init_bias, patterns, targets,
total_epochs, learning_rate, tablename)


% must be in project folder
% cd('')

weights = init_weights;
bias = init_bias;


% inputs are a 20x25 matrix. each column is a different pattern



[~,total_patterns] = size(patterns);

% preallocate tracking matrix
[~, sizey] = size(weights);
weights_history = zeros(25 * total_epochs, sizey, "double");
```

```
update_cycles = 0;
weight_adjustments = 0;


for currentepoch = 1:total_epochs

  for c = 1:total_patterns
    input = patterns(:,c);
    input;

    prediction = activate_perceptron( weights, bias, input );

    error = targets(:,c) - prediction;
    if error ~= 0
      weight_adjustments = weight_adjustments + 1;
    end

    % adjust weights
    [m,n] = size(weights);
    for j = 1:n
      old_weight = weights(:,j);

      new_weight = old_weight + learning_rate * error * input(j);
      weights(:,j) = new_weight;


    end

    % record result
    index = c + total_patterns * (currentepoch - 1);
    weights_history(index,:) =  weights_history(index,:) + weights;

    % adjust bias
    bias = bias + learning_rate * error;
    update_cycles = update_cycles + 1;
  end
end

update_cycles
weight_adjustments

figure;
plot(weights_history, 'LineWidth', 1.5);
```

```
xlabel('Time Step');
ylabel('Value');
title(tablename);
labels = "weight " + (1:20);
legend(labels, 'Location', 'bestoutside');
grid on;
```

```
function [weights, bias, MSE] = train_adaline(init_weights, init_bias, patterns, targets,
total_epochs, learning_rate, modelname)


% must be in project folder
% cd('')

weights = init_weights;
bias = init_bias;


% inputs are a 20x25 matrix. each column is a different pattern



[~,total_patterns] = size(patterns);

% preallocate tracking matricies
[~, sizey] = size(weights)
weights_history = zeros(25 * total_epochs, sizey, "double");
errors = zeros(total_patterns,1);
MSE = zeros(total_epochs,1,"double");

update_cycles = 0;
weight_adjustments = 0;

for currentepoch = 1:total_epochs

   for c = 1:total_patterns
      input = patterns(:,c);
      weighted_sum = 0;

      [prediction, weighted_sum] = activate_adaline( weights, bias, input );
```

```
    error = targets(:,c) - weighted_sum;
    if error ~= 0
      weight_adjustments = weight_adjustments + 1;
    end

    % adjust weights
    [m,n] = size(weights);
    for j = 1:n
      old_weight = weights(:,j);

      new_weight = old_weight + learning_rate * error * input(j);
      weights(:,j) = new_weight;


    end

    % record result
    index = c + total_patterns * (currentepoch - 1);
    weights_history(index,:) =  weights_history(index,:) + weights;
    errors(c) = error;


    % adjust bias
    bias = bias + learning_rate * error;
    update_cycles = update_cycles + 1;
  end

  mse_currentepoch = mean(errors .^ 2);
  MSE(currentepoch) = mse_currentepoch;
end

update_cycles
weight_adjustments

figure;
plot(weights_history, 'LineWidth', 1.5);

xlabel('Time Step');
ylabel('Value');
title( modelname + " (a = " + learning_rate + ") weights");
labels = "weight " + (1:20);
legend(labels, 'Location', 'bestoutside');
grid on;
```

```
% perceptron function with heavyside activation function
function [value] = activate_perceptron(weights, bias, input)

% calculate weighted sum
weighted_sum = weights * input + bias;

% calculate activation
if weighted_sum >= 0
  value = 1;
else
  value = -1;
end
```

```
% perceptron function with heavyside activation function
function [value, weighted_sum] = activate_adaline(weights, bias, input)

% calculate weighted sum
weighted_sum = weights * input + bias;

% calculate activation
if weighted_sum >= 0
  value = 1;
else
  value = -1;
end
```

```
function [results, test_hitrate] = perceptron_test(weights, bias, test_data, ground_truth,
adaline)


  [~,test_rowsize] = size(test_data);
  correct = 0;
  incorrect = 0;
  false_positive = 0;
```

```
  false_negative = 0;
  results = zeros(size(ground_truth));

  for pattern = 1:test_rowsize
    input = test_data(:,pattern);

    if adaline
      prediction = activate_adaline(weights, bias, input);
    else
      prediction = activate_perceptron(weights, bias, input);
    end

    %
    if prediction >= 0 & ground_truth(pattern) >= 0 | prediction < 0 & ground_truth(pattern)
< 0
      correct = correct + 1;
      results(pattern) = 1;
    end

    if prediction >= 0 & ground_truth(pattern) < 0 % false positive
      incorrect = incorrect + 1;
      false_positive = false_positive + 1;
      results(pattern) = -1;
    end

    if prediction < 0 & ground_truth(pattern) >= 0 % false negative
      incorrect = incorrect + 1;
      false_negative = false_negative + 1;
      results(pattern) = -1;
    end

  end

  % HR = Hit Ratio = # correct classifications / # attempted classifications = # correct
classifications / 25
  test_hitrate = correct / test_rowsize;
  false_positive;
  false_negative;
```

```
function [] = test_table_display(ATST1_results, ATST1_hitrate, ATST2_results,
ATST2_hitrate, ATST3_results, ATST3_hitrate, filename)

   TA1matrix = [ATST1_results, ATST1_hitrate; ATST2_results, ATST2_hitrate; ATST3_results,
ATST3_hitrate;];
   numCols = size(TA1matrix, 2);
   colNames = "p" + compose("%02d", 1:(numCols-1));
   colNames(numCols) = "Hit Rate";
   TA1 = array2table(TA1matrix,"RowNames",{'Test 1','Test 2','Test 3'}, "VariableNames",
colNames)



   writetable(TA1, filename, 'WriteRowNames', true);
```

```
function [] = graph_meansquarederrors(mse1, learningrate1, mse2, learningrate2, mse3,
learningrate3, letter)



   total_epochs = length(mse1);
   epochs = 1:total_epochs; % X-axis values

   % --- Plotting ---
   figure;
   hold on; % Keep all lines on the same chart

   % Plot each line with distinct markers and colors
   plot(epochs, mse1, '-o', 'Color', [0 0.4470 0.7410], 'LineWidth', 2, 'MarkerSize', 4);
   plot(epochs, mse2, '-s', 'Color', [0.8500 0.3250 0.0980], 'LineWidth', 2, 'MarkerSize', 4);
   plot(epochs, mse3, '-^', 'Color', [0.9290 0.6940 0.1250], 'LineWidth', 2, 'MarkerSize', 4);

   % --- Chart Styling ---
   title('Adaline ' + letter + ' Mean Squared Error (MSE) Comparison Across Epochs',
'FontSize', 12, 'FontWeight', 'bold');
   xlabel('Epoch', 'FontSize', 10);
   ylabel('Mean Squared Error', 'FontSize', 10);

   % Add a grid for readability
   grid on;

   % Label your specific lines in the legend
   label1 = ['a = ', num2str(learningrate1)];
   label2 = ['a = ', num2str(learningrate2)];
```

```
label3 = ['a = ', num2str(learningrate3)];

lgd = legend(label1, label2, label3, 'Location', 'northeast');
title(lgd, 'Learning Rate');

hold off;
```